

JULIA

История · особенности · запуск · типы данных · функции · multiple dispatch

Дмитриенко Яна

Шабанова Арина

Группа 5030102/30401

Курс «Новые языки программирования»

История Julia

СОЗДАТЕЛИ

Jeff Bezanson, Stefan Karpinski, Viral B. Shah, Alan Edelman

MIT, 2012

МОТИВАЦИЯ

Решить **проблему двух языков** в научных вычислениях

ЛИЦЕНЗИЯ

Open source, MIT license

НАЗВАНИЕ

По предложению друга создателей

2012

Первый коммит

Начало разработки языка

2012

Публичное объявление

Официальный анонс языка

2018

Julia 1.0

Стабильный релиз (август)

2024

Julia 1.11

LTS 1.10, stable 1.11

Проблема двух языков

В научных вычислениях исследователи часто используют **два языка**:

- **Прототипирование** — Python / MATLAB / R (удобно, но медленно)
- **Вычисления** — C / C++ / Fortran (быстро, но сложно)

ПОСЛЕДСТВИЯ

Двойное обслуживание кодовой базы

Ошибки при переводе между языками

Барьер для экспертов предметной области



Ключевые особенности Julia

Multiple Dispatch

Основная парадигма выбора методов по типам всех аргументов

JIT через LLVM

Производительность, близкая к C

Динамическая типизация

С опциональными аннотациями типов

Простой синтаксис

Похож на Python / MATLAB

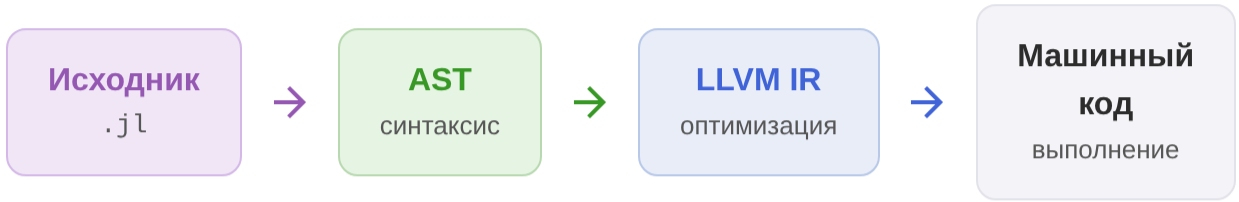
Массивы и линейная алгебра

Built-in поддержка из коробки

Metaprogramming

Макросы и генерация кода

Модель выполнения: JIT и LLVM



- Julia — **не интерпретируемый** и **не традиционный AOT-компилируемый** язык
- При первом вызове с конкретными типами аргументов Julia **компилирует специализированную версию** функции
- Последующие вызовы используют **кэшированный код** со скоростью, близкой к C

Проблема TTFP (time-to-first-plot): первый вызов функции может быть заметно медленнее из-за накладных расходов компиляции

ПРИМЕР: ПЕРВЫЙ И ПОВТОРНЫЙ ВЫЗОВ

```
# Первый вызов — компиляция println("First call...") @time sqrt(2.0) # Повторный вызов — кэш println("Second call...") @time sqrt(2.0)
```

```
First call...
  0.055085 seconds (117.26 k allocations: 6.979 MiB)
Second call...
  0.000003 seconds (1 allocation: 16 bytes)
```

Второй вызов выполняется на порядки быстрее — используется скомпилированный код

Запуск Julia

Локально

Установка с `julialang.org`

```
# Проверка версии julia --
version # Запуск скрипта julia
script.jl # Интерактивный REPL
julia>
```

REPL — интерактивная оболочка

Docker

Официальный образ: `julia:1.11`

```
# REPL в Docker docker run -it
julia:1.11 # Скрипт в Docker
docker run -it julia:1.11 \
julia script.jl # Сборка из
Dockerfile docker build -t
myapp .
```

Docker Hub: julia

Онлайн

Без установки, в браузере

JuliaHub

`juliahub.com`

Google Colab

Jupyter + Julia kernel

Replit

Облачная IDE с Julia

Быстрый старт без установки

Julia в Docker

DOCKERFILE

```
FROM julia:1.11 WORKDIR /app COPY . . CMD ["julia", "main.jl"]
```

ТЕРМИНАЛ

```
# Сборка образа $ docker build -t julia-app . # Запуск контейнера $ docker run --rm julia-app
```

ИНТЕРАКТИВНЫЙ REPL

```
$ docker run -it julia:1.11 _ _ _ _(_)_ | Documentation: https://docs.julialang.org (_) | (_) (_) | _ _ _| |_ __ _ | Type "?" for help, "]?" for Pkg help. | | | | | | |/ _` | | | | |_| | | | (_| | | Version 1.11.x _/ |\__'_|_|_|\__'_| | |__/ | julia>
```

Официальный образ

- Образ `julia` на Docker Hub
- Тег `1.11` — текущий стабильный
- `-it` — интерактивный режим с TTY

Первая программа

HELLO.JL

```
println("Hello, Julia!")
```

```
Hello, Julia!
```

REPL-СЕССИЯ

```
julia> 2 + 3 5 julia> println("Hello!") Hello! julia>
6 * 7 42
```

Расширение файлов

Julia-файлы используют расширение `.jl`

REPL

Интерактивная оболочка — читай-вычисляй-выводий цикл. Запускается командой `julia`

Автовывод

Последнее выражение в REPL выводится автоматически — без `println`

Типы данных и переменные

ОБЪЯВЛЕНИЕ ПЕРЕМЕННЫХ

```
x = 42 y = 3.14 name = "Julia" flag = true ch = 'J'
```

TYPEOF() — ОПРЕДЕЛЕНИЕ ТИПА

```
typeof(x) # Int64 typeof(y) # Float64 typeof(name) # String
typeof(flag) # Bool typeof(ch) # Char
```

Динамическая типизация

Типы определяются в момент выполнения, но система типов богата и строга

Переменные — привязки

Переменная — это привязка к значению, а не контейнер. Нет ключевого слова для объявления

Соглашения об именах

`snake_case` для переменных, `PascalCase` для типов

Платформозависимость

`Int64` на 64-битных, `Int32` на 32-битных системах

Операции и строки

АРИФМЕТИКА

```
10 ÷ 3 # 3 (целочисленное) 10 / 3 # 3.333...
(вещественное) 2^10 # 1024 (степень) 17 % 5 # 2 (остаток)
```

СТРОКИ

```
"Hello " * "Julia" # "Hello Julia" (конкатенация)
"Julia"^3 # "JuliaJuliaJulia" (повторение) name = "Julia"
println("Hello, $name!") # Hello, Julia! (интерполяция)
```

СРАВНЕНИЕ И ЛОГИКА

```
# Сравнение: == != < > <= >= # Логические: &&
|| !
```

! Ключевое отличие

В Julia два оператора деления:

/	÷
Вещественное	Целочисленное
10/3 → 3.333	10÷3 → 3

Преобразование типов

КОНСТРУКТОРЫ ТИПОВ

```
Int64(3.0) # → 3 Float64(5) # → 5.0 typeof(Int64(3.0)) # → Int64
convert(Float64, 5) # → 5.0
```

СТРОКИ И ЧИСЛА

```
parse(Int, "42") # → 42 string(42) # → "42"
```

АВТОМАТИЧЕСКОЕ ПРОДВИЖЕНИЕ

```
2 + 3.0 # → 5.0 (Float64) promote(2, 3.0) # → (2.0, 3.0)
```

Конструктор vs convert

`T(x)` — конструктор, `convert(T, x)` — безопасное преобразование

InexactError

`Int64(3.14)` — ошибка! Нельзя точно представить в Int64. Но `Int64(3.0)` работает

Продвижение типов

При смешанных операциях Julia автоматически приводит типы к общему: Int + Float64 → Float64

Коллекции

МАССИВЫ

```
a = [1, 2, 3] # Vector{Int64} matrix = [1 2; 3 4] # 2×2 Matrix
a[1] # → 1 (1-индексация!) a[1:2] # → [1, 2] push!(a, 4) # →
[1,2,3,4]
```

1-индексация

Индексы начинаются с 1 — для математической согласованности

Суффикс !

Функции, изменяющие аргументы, заканчиваются на ! (push!, sort!)

КОРТЕЖИ

```
t = (1, "hello", 3.14) t[1] # → 1 (неизменяемые)
```

СЛОВАРИ

```
d = Dict("a" => 1, "b" => 2) d["a"] # → 1
```

ДИАПАЗОНЫ

```
1:5 # 1, 2, 3, 4, 5 1:2:10 # 1, 3, 5, 7, 9
```

Области видимости

Глобальная, локальная и жёсткая локальная области

SCOPE_DEMO.JL

```
x = 10 # глобальная function demo() x = 20 # локальная println("Inside: $x") end demo() # → Inside: 20 println("Outside: $x") # → Outside: 10
```

HARD_LOCAL.JL

```
for i in 1:3 x = i # новая локальная end # x не видна за пределами цикла let y = 100 # жёсткая локальная область # не наследует от внешней end
```

Глобальная область

Переменные на уровне модуля. Доступны из любого места внутри модуля.

Локальная область

Внутри функций, циклов, структур. Функции всегда создают свою область видимости.

Жёсткая локальная область

Блоки `let` не наследуют переменные из внешней области. Циклы внутри функций — жёсткая область.

Ключевое правило: циклы создают «мягкую» локальную область в модулях, но «жёсткую» внутри функций.

Функции

Объекты первого класса с гибким синтаксисом

ПОЛНАЯ ФОРМА

```
function add(x, y) x + y end
```

ОДНОСТРОЧНАЯ

```
add(x, y) = x + y
```

АНОНИМНАЯ

```
add = (x, y) -> x + y
```

АРГУМЕНТЫ ПО УМОЛЧАНИЮ И КЛЮЧЕВЫЕ

```
f(x, y=10) = x + y # по умолчанию g(x; power=2) = x^power
# ключевое h(args...) = sum(args) # varargs
```

ФУНКЦИЯ КАК АРГУМЕНТ

```
map(sqrt, [1, 4, 9])
```

```
 [1.0, 2.0, 3.0]
```

★ Объекты первого класса

Функции можно передавать как аргументы, сохранять в переменные и возвращать из других функций.

Автоматический возврат

Последнее выражение в функции возвращается автоматически. Ключевое слово `return` — необязательно.

Multiple dispatch

Метод выбирается по типам **всех** аргументов. Подробнее — на следующем слайде.

Гибкость аргументов

Значения по умолчанию, именованные аргументы через `;`, переменное число аргументов через `...`

Multiple Dispatch

Ключевая парадигма Julia — выбор метода по типам **всех** аргументов

ТИПЫ И СТРУКТУРЫ

```
abstract type Shape end struct Circle <: Shape radius::Float64
end struct Rectangle <: Shape width::Float64 height::Float64 end
```

МЕТОДЫ AREA()

```
area(c::Circle) = π * c.radius^2 area(r::Rectangle) = r.width *
r.height
```

ИСПОЛЬЗОВАНИЕ

```
area(Circle(5.0)) area(Rectangle(3.0, 4.0))
```

```
78.53981633974483
12.0
```

ДИСПЕТЧЕРИЗАЦИЯ ПО ДВУМ АРГУМЕНТАМ

```
intersect(s1::Circle, s2::Rectangle) = "○∩□"
intersect(s1::Rectangle, s2::Circle) = "□∩○"
```

Single vs Multiple Dispatch

Один аргумент

obj.method(arg)
Тип **obj** определяет метод

Все аргументы

func(a, b)
Типы **a** и **b** определяют метод

Открытая расширяемость

Новый тип + новые методы — без изменения существующего кода. Добавьте `Triangle` и `area(t::Triangle)` — готово.

Диспетчеризация по всем аргументам

`intersect(Circle, Rectangle)` и `intersect(Rectangle, Circle)` — разные методы.

Производительность: Julia компилирует специализированную версию для каждой комбинации типов — скорость близка к статическому полиморфизму.

Broadcasting

Точечный синтаксис для поэлементных операций

ПОЭЛЕМЕНТНЫЕ ОПЕРАЦИИ

```
x = [1, 2, 3] x .^ 2 x .+ 10
```

```
[1, 4, 9]
[11, 12, 13]
```

ПОЛЬЗОВАТЕЛЬСКАЯ ФУНКЦИЯ

```
f(x) = x^2 + 1 f.(x)
```

```
[2, 5, 10]
```

ПРИСВАИВАНИЕ НА МЕСТЕ

```
y = [0, 0, 0] y .= x .+ 1 # мутация на месте
```

СЛИЯНИЕ ТОЧЕК (DOT FUSION)

```
2 .* x .+ 1 # один цикл!
```

Точка = поэлементно

Добавьте `.` перед оператором или функцией — она применяется к каждому элементуА коллекции.

Явный контроль

В MATLAB/NumPy векторизация автоматическая. В Julia вы явно выбираете, когда применять поэлементно — больше контроля.

Слияние (dot fusion)

Julia объединяет несколько точечных операций в **один цикл** для производительности: `2 .* x .+ 1` — без промежуточных массивов.

Запомните: точка `.` = поэлементно, явно, с автоматической оптимизацией слияния.

Итоги

Ключевые выводы о языке Julia

1 **Проблема двух языков решена**

Один язык для прототипирования и высокопроизводительных вычислений

2 **JIT через LLVM**

Скорость близкая к C при динамической типизации

3 **Multiple dispatch**

Выбор метода по типам всех аргументов — открытая расширяемость

4 **Broadcasting**

Явный поэлементный синтаксис с автоматической оптимизацией (fusion)

5 **Функции первого класса**

От многострочных до однострочных, анонимных и с varargs

6 **Система типов**

Абстрактные типы, конкретные типы, параметрические типы

Спасибо за внимание!

Курс «Новые языки программирования»